

Memo — Constrained Document Layout Detection & Reasoning API

RT-DETR-L fine-tuned on DocLayNet, evaluated on a held-out test split

[GitHub repository](#) · [Live demo](#) · [Evaluation report](#)

1. Domain, dataset, and sourcing

I chose document layout analysis — detecting structural regions (tables, headings, captions, formulas) on a page — over traffic or PPE for three reasons. First, all 11 classes are non-COCO, so no pretrained checkpoint contributes any score; PPE and traffic both lean on COCO's person/car for a chunk of their apparent performance. Second, it yields a genuinely *architectural* "insufficient information" case for Part B, not a tuned threshold. Third, it's adjacent to what RAP builds — RAPFlow extracts from unstructured documents, and layout detection is the perception stage underneath that.

Dataset: [pierreaguillou/DocLayNet-base](#), a 10% subset of IBM Research's DocLayNet — 8,057 pages, CDLA-Permissive-1.0, spanning 6 document categories (financial reports, scientific articles, laws, tenders, manuals, patents). Labels are expert-annotated by the original authors; I did not relabel, so label quality is inherited rather than claimed. I chose the subset over the full 80k-page release purely for the time budget (~30 GB, ~45 min just to download).

One real data hazard: `bboxes_block` repeats each region's box once per text line, so a naive read of a 40-region page yields ~1,100 duplicate annotations. [scripts/prepare_dataset.py](#) deduplicates on (`box`, `category`), drops degenerate boxes, and reports both counts per split. Because the class-index base was inferred rather than documented, the script also renders sample pages with decoded labels drawn on — an off-by-one there would be invisible to every automated check, so I verified it by eye before spending GPU time.

2. Split strategy

I used the author's published split (6,910/648/499) rather than re-stratifying. That is the reproducible choice and keeps numbers comparable to published DocLayNet work, but it means inheriting whatever leakage it has. I measured rather than assumed: **0% of test pages share a source PDF with training** ([metrics.json](#), `split_leakage`). DocLayNet pages come from multi-page PDFs, so had that been non-zero my test scores would have been measuring memorisation as much as generalisation.

3. Evaluation — and what it does *not* tell you

15 epochs, RT-DETR-L, Tesla T4, 2.27h. Test set = 499 held-out pages, 6,349 instances.

Class	Precision	Recall	mAP50	mAP50-95
Page-footer	0.865	0.853	0.843	0.496
Text	0.804	0.781	0.832	0.658
Formula	0.676	0.760	0.788	0.412
Section-header	0.745	0.734	0.762	0.485
Table	0.763	0.730	0.761	0.633
List-item	0.705	0.632	0.671	0.452
Page-header	0.712	0.659	0.658	0.386
Caption	0.572	0.644	0.622	0.411
Picture	0.416	0.622	0.495	0.380
Title	0.503	0.192	0.224	0.125
Footnote	1.000	0.048	0.182	0.096
Overall	0.706	0.605	0.622	0.412

I report per-class and per-category numbers because one aggregate mAP cannot answer what I need to know. Per-category mAP50 spans 0.495 (`laws_and_regulations`) to 0.745 (`scientific_articles`) — a direct signal about which document styles generalise. I also measured RT-DETR's query-budget saturation rather than letting it hide inside recall: **0% of pages exceeded the 300-query budget** (max 95 regions), so saturation explains none of these results.

What mAP does not tell you: it measures box localisation and classification only — nothing about reading order, and nothing about whether a detected Table's internal structure is recoverable downstream. A model can score well here and still be useless for actually extracting a table's rows. Inference is 41.3 ms/image on the T4.

4. Five failure cases

1. Footnote — recall collapse (0.048, mAP50 0.182). I predicted thin classes would suffer at 640px. Only half right: Footnote has 47 instances across 18 images. Rarity, not size, dominates — see #2.

2. Page-footer **should have failed identically, and didn't (mAP50 0.843)**. Equally thin, same page-margin position, but 408 instances (~9x) and a highly consistent location. This is the most useful negative result in the run: it revised my hypothesis from "thin objects fail" to "thin *and rare* objects fail; thin-but-frequent-and-positionally-consistent ones are fine."

3. **Dense repeated-entry layouts produce duplicate, jittery boxes** (image; worst page in the run — 38 missed, 82 false positives). A bank's org directory: ~25 near-identical Section-header/Text pairs in two columns. Ground truth has one clean box per entry; predictions overlap and offset at scattered confidence (0.25–0.85). At 640px the model cannot cleanly separate many near-identical tightly-packed instances — a fixed-query architecture limit on repetitive layouts, not a training bug. The same pattern recurs on a scientific article's reference list and on individual formula blocks (image), confirming it generalises across document types.

4. **Composite Picture regions fragment into overlapping sub-boxes** (image; patents: 0 missed, 0 misclassified, 61 false positives — the cleanest signature in the run). Each ground-truth Picture is one chemical diagram; the model emits 2–3 overlapping boxes per diagram, treating the sub-drawing and inline formula text as separable. This directly explains Picture's precision (0.416) against its recall (0.622) — it finds pictures, just not as single coherent regions.

5. Page-header **confused with Text on running headers** — found live on a random unseen legal-code page, not from the mined set: a running citation line was labelled Text at 0.34–0.46, well under the model's confident paragraph-level Text calls (0.68–0.96). The low confidence is itself informative — the model is uncertain exactly where it should be classifying differently.

5. Part B — routing, and the honest refusal

The router (regex prefilter → Groq, strict json_schema constrained decoding) sets needs_detection *before* anything else runs. Its prompt states the detector's vocabulary is layout-only: it knows *where* regions are, never *what* text says. So "what is the invoice total" sounds document-related but is content, and routes to out_of_scope without ever calling the detector. On any LLM error it fails closed to out_of_scope rather than guessing.

Live-tested example. Asked "What law is this document about?" against a real test page:

"I can't answer that from this image's layout: Question asks about document content, which the detector cannot read."

insufficient_information=True is set in code from the router/guardrail state, never by asking the LLM how confident it feels — a confident-sounding wrong answer is one bad prompt away otherwise. The deterministic guardrail (no detections / all below a 0.50 floor / >30% of a count in the ambiguous 0.25–0.50 band / query saturation) is what decides, and fluent synthesis text cannot talk it out of triggering. Every response carries a full reasoning trace for auditing.

6. Mid-project pivots

- **PPE → traffic → document layout.** PPE was my first instinct, but person is a COCO class, so part of the score would be inherited rather than earned; traffic was worse on the same axis. Documents make every mAP point attributable to fine-tuning.
- **Full DocLayNet → the 10% subset.** ~30 GB and ~45 min of download was not defensible against a one-day window for identical annotation quality.
- **OpenAI → Groq.** Groq supports json_schema with strict: true, so the router's output is *structurally* guaranteed to match schema rather than prompted-and-hoped-for.
- **30 → 15 epochs.** An interactive Kaggle session was recycled mid-run, wiping the dataset and every checkpoint through epoch 10 (already at mAP50 0.69). I rebuilt with a halved epoch budget to fit the remaining window, and switched to committed runs. Noting this rather than hiding it: the final numbers are from a 15-epoch run, not a 30-epoch one.
- **Streamlit → Gradio for hosting.** HF's current Space flow offers only Gradio/Docker/Static; Docker needs a paid plan, and free Gradio Spaces run on ZeroGPU. Both UIs are in the repo.

7. Engineering: containerisation, deployment, logging, error handling

- **Docker** — CPU-only torch wheel (no CUDA bloat in an inference-only image), non-root user, HEALTHCHECK against /health. Built and actually run, not just written: verified it starts cleanly with weights *absent* and reports model_loaded: false rather than crashing.
- **Deployed** — live on [Hugging Face Spaces](#) (Gradio on ZeroGPU). Weights are 66 MB, too large for the repo, so the app fetches them from a version-tagged GitHub Release on cold start via MODEL_URL — no manual copying per host.
- **Structured logging** — one JSON line per request with request_id, endpoint, latency_ms, detection count, and which guardrail rule fired, so "how often did X happen" is a grep rather than a re-run. Model timing is measured separately from request overhead.
- **Error handling** — 413 oversized upload, 415 wrong content-type or undecodable image (the header alone isn't trusted), 422 missing/empty question, 503 model not loaded (a known operational state, not a crash), and a global handler that logs the real exception server-side while returning a safe message — no traceback ever reaches the caller.